UNIT 3 • SOFTWARE DESIGN Fast, clear study notes

Read to understand—not to memorise long paragraphs. This guide follows the six topics in your PPT. Repeated slides are combined, decorative images are removed, and explanations stay short. Read the meaning, the example, and the “Remember” line for each topic.

Your reading map

Page	What you will learn
2	Software design: meaning, importance, goals and work products
3	Design concepts, principles and the restaurant example
4	Requirements models → four design models
5	Good design and FURPS quality attributes
6	Component design: function-oriented versus object-oriented
7	Coupling: dependency between modules
8	Cohesion: related work inside a module
9	Architecture: the three-layer example
10	UI design: give the user control
11	UI design: reduce memory load and stay consistent
12	WebApp design: all six design areas
13	Last-minute revision and answered practice

How to use these notes

• First reading: understand the short explanation and example.

• Second reading: revise the bold terms and comparison tables.

• Before the exam: use the last page and practise explaining topics aloud.

Source: unit-3.pptx, 47 slides, Software Engineering, DU - 2304CS304, Prof. Mayur Padia. Teaching examples and memory tips are added explanations. Questions are practice—not predictions.

Scope note: separate files linked inside the PPT were not supplied. Their unseen contents are not included. This version focuses on the academic text in the PPT, not a reproduction of every decorative illustration.

1. Software design: the basic idea

Definition: Software design means converting requirements into a plan for building software.

In easy words: Before writing code, decide what parts the software needs, what data it stores, how parts communicate, and what steps each part follows. This plan is called a **blueprint**.

Example: A student portal must show marks. Its design decides that a screen requests marks from a service, which gets the correct records from storage.

Why is design important?

• It helps you understand the system before coding.

• You can review and improve the plan early, when changes are easier.

• It connects user needs with an implementable solution.

• Without clear design, small changes can cause failures; testing is harder; problems may be discovered late, after time and money are spent.

Five design work products

Item	What must be decided?
Modules	Which separate parts are needed? Example: login, marks, payment.
Call/control relationships	Which part calls which other part? This is also called an invocation relationship.
Interfaces	Exactly what data the parts exchange. An interface is an interaction contract, not always a screen.
Algorithms	The ordered steps each part follows.
Data structures	How each part organises and stores its information.

Three goals of design

Goal	Easy meaning
Firmness	Works reliably; avoids defects that prevent correct operation.
Commodity	Does the job for which it was intended.
Delight	Feels pleasant and easy to use.

Two thinking steps

Diversification: collect different ideas and possible solutions from books, tools and experience.
Convergence: compare them, reject weaker choices, and select those that fit the requirements.

Remember: Requirements = what is needed. Design = how you plan to build it. Design helps assess quality early; testing is still necessary.

2. Design concepts and principles

Concepts are ideas used to understand software structure. **Principles** are guidelines that help you make good design decisions.

Named concepts: quick explanations

Concept	Meaning + simple example
Abstraction	Focus on what matters; hide unnecessary detail. A pay() operation hides bank-processing steps.
Modularity	Divide software into manageable parts. Login and payment are separate modules.
Encapsulation	Keep related data and operations behind a controlled interface. Use deposit() rather than editing a balance directly.
Inheritance	A specialised class receives features from another class. A savings account uses common account features.
Polymorphism	Different objects respond to the same operation in their own way. Different payment types implement pay().
Cohesion	How closely related the work inside one module is. Explained on page 8.
Coupling	How dependent different modules are on each other. Explained on page 7.
Design pattern	A reusable approach to a recurring design problem—not a complete finished program.

Three example principles from the PPT

- **Keep the design simple:** avoid unnecessary parts and complicated interactions.
- **Separate concerns:** keep different responsibilities separate, such as UI and storage.
- **Encapsulate what varies:** isolate likely changes behind a stable interface, such as a replaceable payment provider.

Restaurant analogy: understand the levels

Restaurant example	Software design activity
Decide customers, menu and goals.	Conceptual design: understand goals, users and key features.
Plan kitchen, dining area and storage.	Architecture: arrange major parts, such as UI, services and database.
Specify how a cooking station works.	Component design: detail one part’s logic, data and interactions.
Create a clear ordering menu.	Interface design: make interaction user-friendly.
Build and trial the restaurant.	Implementation and testing: code and check the software.

PPT clarification: slide 3 repeats “Conceptual Design” beside a description of detailed module internals. That description matches component-level design in slide 16.

Remember: Concept = an idea. Principle = advice for applying ideas well.

3. From requirements to design

Design begins after requirements have been analysed. The **requirements model** describes user needs, relevant entities and expected behaviour. It supplies information for the design models.

Three requirements-model categories

Category	Tools named in PPT	What they show
Scenario-based	Use cases; activity diagrams	User tasks and workflows.
Class-based	Class diagrams; CRC models	Entities, responsibilities and relationships.
Behavioural	State diagrams; sequence diagrams	State changes and ordered interactions.

CRC = Class–Responsibility–Collaborator. A class represents a kind of entity; its responsibility is what it knows or does; its collaborators are the other classes it works with.

Example: an Order class calculates a total and collaborates with Payment.

Four design models: the main outputs

Model	What it plans	Simple portal example
Data/class design	Implementable classes and data structures. Uses class models and CRC objects/relationships.	Student and Marks records.
Architectural design	Major parts, their relationships, and suitable styles/patterns.	UI → service → data access.
Interface design	Information exchange with people and other systems. Uses scenarios and behavioural models.	Marks screen and its service request.
Component-level design	Detailed logic inside a component. Uses class-based and behavioural models.	Steps to check access and retrieve marks.

How the pieces connect

Requirements → Design → Coding → Testing

The PPT highlights three design steps: represent the architecture, model the interfaces, and design the components. Data/class design is also part of the full design model. These activities influence each other; they are not isolated boxes.

Do not confuse these

- **Architecture:** overall structure. **Component:** details inside one part.
- **Interface:** interaction with a user or another system—not just colours and buttons.
- **Algorithm:** procedure. **Data structure:** organisation of information.

Remember: The four outputs are Data/classes, Architecture, Interfaces and Components.

4. Good design and FURPS

Three basic quality guidelines

• Meet **explicit requirements** (clearly stated needs) and accommodate **implicit requirements** (stakeholder expectations).

• Be readable and understandable for developers, testers and support staff.

• Give a complete implementation view of **data, functionality and behaviour**.

Characteristics of good design

A good design uses recognisable architectural styles/patterns and well-designed components; supports gradual implementation and testing; divides the system into modules; and keeps distinct views of data, architecture, interfaces and components.

It also chooses suitable data structures, gives components independent responsibilities, reduces interface complexity, follows a repeatable method, and uses notation that communicates clearly.

FURPS: five ways to assess quality

The PPT attributes this quality model to Hewlett-Packard.

Attribute	Easy meaning	Assessment points in PPT
F — Functionality	Does it provide the needed features?	Features, capabilities, generality of functions, security.
U — Usability	Is it easy to understand and use?	Human factors, appearance, consistency, documentation.
R — Reliability	Does it work correctly and dependably?	Failure frequency/severity, output accuracy, MTTF, recovery, predictability.
P — Performance	How fast and efficiently does it work?	Processing speed, response time, resources, throughput, efficiency.
S — Supportability	Is it easy to extend and maintain?	Extensibility, adaptability, serviceability.

Terms you should understand

• **MTTF:** mean time to failure—average operating time before failure.

• **Response time:** waiting time for one request. **Throughput:** amount of work completed per time interval.

• **Extensibility:** ease of adding features. **Adaptability:** ease of adjusting to change. **Serviceability:** ease of diagnosing and maintaining software.

Example: a marks portal that gives correct results is reliable; one that responds quickly performs well; one with clear labels is usable. These qualities are different.

Remember: FURPS = Functionality, Usability, Reliability, Performance, Supportability.

5. Component-level design

Component: a software building block with a clear job. The PPT describes it as modular, deployable and replaceable.

Component-level design: specify that part's internal logic, interfaces and interactions. It supports reuse, modularity and maintenance. Independent testing is easier when the boundary is clear; independent deployment depends on how the part is packaged.

Example: a login component receives credentials, checks them, and reports success or failure. It should not also control unrelated tasks such as theme settings.

Function-oriented design

Organise software around **functions or procedures that operate on data**. Split a large task into smaller reusable operations. This suits clear **input → process → output** work.

Example: read marks → validate marks → calculate result → display result. Each operation is a separate function.

Tool: a Data Flow Diagram (DFD) shows how data moves between processes, stores and outside entities.

Object-oriented design (OOD)

Organise software around **objects**. Each object bundles **attributes** (data) and **methods** (operations). A **class** defines the kind of object; an object is one instance of it.

Example: Student has marks as data and calculateResult() as an operation. Different Student objects hold different students' marks.

OOD focuses on classes and their interactions. It promotes encapsulation, inheritance and polymorphism. **Tools:** UML class diagrams and sequence diagrams. UML means Unified Modeling Language.

The difference in one table

Point	Function-oriented	Object-oriented
Main focus	Tasks/functions.	Objects/classes.
Data and operations	Functions operate on supplied data.	Related data and methods are bundled.
Breakdown	Split the work into smaller operations.	Identify entities and collaborations.
Source tools	DFDs.	UML class and sequence diagrams.
Example notation	calculateTotal(marks)	student.calculateTotal()

What the notation means

In calculateTotal(marks), marks is passed into a function. In student.calculateTotal(), the method belongs to the student object and can use its data. These are illustrative calls, not full programs from the PPT.

Remember: Function-oriented = organise by tasks. Object-oriented = organise by entities with data + behaviour.

6. Coupling: between modules

Definition: coupling is the degree of dependency between software modules.

Easy meaning: how much does one part depend on another? If a change in storage forces changes throughout the UI, the parts are tightly coupled. **Lower coupling is generally better.**

Source example: screen and storage

High coupling	Lower coupling
MainActivity directly uses DatabaseHelper and SharedPreferences.	MainActivity → Repository → data sources.
The screen knows storage details; a storage change may affect it.	The Repository hides storage details behind a suitable interface.
UI and storage are harder to change independently.	Local database or cloud API can change while the UI contract stays stable.

Terms: MainActivity is an Android screen/controller; SharedPreferences stores small key–value settings. A Repository is an intermediary for data access. An API lets software request another system's functionality.

Two coupling types in the PPT

Type	Meaning	Example
Data coupling	Pass only the necessary data.	saveUser(email: String, password: String)
Stamp coupling	Pass a composite structure; the PPT warns about passing more than is needed.	Pass the entire User object when only email is required.

Syntax: saveUser is the operation name; email and password are input parameters; String means text. Passing an object is not automatically wrong—check whether the whole object is genuinely required.

All eight benefits of low coupling

Benefit	Why it helps
Maintainability	Change, debug or replace parts with less impact elsewhere.
Modularity	Build, test and maintain parts separately; support reuse.
Scalability	Add/remove modules more easily as the system changes.
Flexibility	Change an individual module with less disruption.
Parallel development	Different teams can work concurrently.
Testability	Test a module in isolation more easily.
Fewer unintended effects	A change is less likely to cause unexpected side effects.
Code organisation	Clear boundaries and responsibilities.

Remember: Coupling = BETWEEN modules. Aim for LOW coupling, not zero communication.

7. Cohesion: inside a module

Definition: cohesion is the degree to which the parts inside a module work together toward one well-defined purpose.

Easy meaning: do all the methods belong together? **Higher cohesion is generally better.** A module can contain several methods as long as they support the same responsibility.

Source examples

High cohesion	Low cohesion
LoginViewModel handles login only.	AppUtilityClass mixes unrelated jobs.
validateInput(), loginUser(), handleLoginResult().	Login, date formatting, API calls, theme settings.
All methods support one purpose: login.	Several different purposes are bundled together.

Improvement: keep login in a login module; put date formatting, API communication and theme settings in suitable separate places. This is an added explanation of the source example.

Three benefits of high cohesion

- **Maintainability:** a focused component is easier to understand, modify or replace.
- **Modularity:** a clear responsibility supports independent development and testing.
- **Reusability:** a focused part is easier to use in another place or project.

Cohesion versus coupling: must-know comparison

Point	Cohesion	Coupling
Where?	Inside one module.	Between modules.
Measures	How related its responsibilities are.	How dependent the modules are.
Preferred direction	HIGH.	LOW.
Poor example	Utility class with unrelated jobs.	Screen tied to storage internals.
How to improve	Separate unrelated responsibilities.	Use narrow, stable interfaces; hide unnecessary details.

One important warning

High cohesion does not automatically mean low coupling. A payment module may do only payment work, yet depend directly on many other modules' internals. Check both qualities separately.

Quick self-check

Question: a class formats dates, logs in users and changes themes. What is the problem? **Answer:** low cohesion, because its tasks are unrelated.

Question: changing a database format forces screen changes. What is the problem? **Answer:** tight coupling between UI and storage details.

Remember: Good design = HIGH cohesion + LOW coupling. Related work stays together; unnecessary dependencies stay out.

8. Architecture and the three-layer model

Definition: software architecture describes the system's structure: its components, their externally visible properties, and their relationships.

Easy meaning: the big-picture arrangement of software parts and how they cooperate. "Externally visible" means what other parts can use or depend on, such as an exposed operation.

Building analogy: rooms, corridors and services form one usable building. Software components and their connections form one working system. The PPT also connects architecture with meeting the owner's purpose and the building's appearance.

Why architecture matters

• It gives stakeholders a shared model for communication.

• It records early decisions that strongly affect later development.

• It helps check requirements and compare options while changes are easier.

• It reduces risk and gives an understandable view of how the system works.

Three layers in the PPT

Layer	Job	Named technology examples
Presentation / UI	Handle user interaction.	Web: HTML, CSS, JavaScript, React, Angular. Mobile: Kotlin (Android), Swift (iOS).
Business logic / service	Apply core rules and processing; provide RESTful APIs.	Java, C#, Python, Node.js.
Data access	Communicate with databases.	SQL; PostgreSQL, MySQL; ORM tools Hibernate, Entity Framework.

Request flow: User → UI → Service → Data access → Database. The result returns through the layers to the UI.

Example: the UI receives a "View marks" click; the service checks which marks the user may see; data access retrieves them; the UI displays the permitted result.

Terms and common confusion

• **ORM:** object–relational mapping—tools connecting program objects with relational database data.

• **Data access is not the database:** it is the code that communicates with it.

• **Architecture versus component design:** overall arrangement versus detailed logic inside a part.

• The PPT calls this three-tier architecture. Its explanation describes logical layers; separate deployment tiers depend on where the software actually runs.

Remember: UI interacts. Service applies rules. Data access handles database communication.

9. UI design: put the user in control

User interface design creates an effective communication medium between a person and a computer.

Identify **interface objects** (buttons, fields, icons) and **actions** (enter, open, delete). Arrange them into a screen layout that supports a **prototype**—an early model of the interface.

The three golden rules

1. Put the user in control.
2. Reduce the user's memory load.
3. Make the interface consistent.

Rule 1: meaning

Users should be able to choose actions, navigate, interrupt tasks and recover from mistakes. Do not trap them in unnecessary steps or make them manage technical details they do not need.

All six principles under rule 1

Principle	Meaning + example
Avoid unwanted interaction modes	A mode changes how input behaves. Spell-check mode should support the requested task, not force unrelated actions.
Allow flexible interaction	Support appropriate keyboard, mouse or voice input.
Allow interruption and undo	Let users stop operations and reverse actions where possible.
Support skill growth and customisation	Provide faster methods for experienced users. A macro saves a sequence of actions for reuse.
Hide technical internals	Casual users should not need operating-system or file-management knowledge unnecessarily.
Allow direct manipulation	Let users work on visible objects—for example, drag a document to a trash icon.

A simple example

While uploading a file, a user can cancel. After an accidental deletion, they can use Undo if supported. They can select an action with a mouse or keyboard. These features give the user control.

Do not misunderstand the rule

"User control" does not mean exposing every internal setting. Nor does it mean every real-world action can be undone. For irreversible actions, clearly explain consequences and use suitable safeguards.

Remember: Control = choice, flexibility, recovery and understandable interaction.

10. UI design: memory and consistency

Rule 2: reduce the user’s memory load

Show what users need rather than making them remember it across screens. More remembering means more chances of error. Use visible information, clear labels and familiar cues.

All five principles	Meaning + example
Reduce short-term memory demand	Show visual cues and previous choices. Keep cart items and total visible.
Use meaningful defaults	Provide sensible starting values and a reset option.
Use intuitive shortcuts	Choose memorable associations: Ctrl+P for Print. A mnemonic is a memory aid.
Use a real-world metaphor	Use familiar ideas such as a checkbook/check register in a bill-payment system.
Reveal detail progressively	Show the overview first; reveal extra detail when needed. This is progressive disclosure.

Rule 3: make the interface consistent

Use uniform layouts, colours, fonts, terminology and behaviour. Familiar patterns help people learn faster and reduce confusion.

All three principles	Meaning + example
Show meaningful task context	Use window titles, icons and consistent colour coding so users know where they are.
Be consistent across a product line	Related applications, such as an office suite, should share interaction rules.
Respect learned expectations	Do not change established behaviour such as Ctrl+S for Save without a compelling reason.

Distinguish the rules quickly

Feature	Main rule illustrated
Cancel or Undo	User control.
Visible cart total or sensible defaults	Reduced memory load.
Same Save behaviour throughout the app	Consistency.

One feature can support more than one rule. For example, a consistent shortcut can also reduce memory demands.

Remember: Memory: help users recognise instead of recall. Consistency: familiar controls should behave predictably.

11. WebApp design: the six areas

WebApp design includes both technical and nontechnical work: appearance, UI layout, overall structure, content, functionality and navigation.

Reviewing the design before extensive content/code production helps improve quality early. Design is important, but later implementation and testing still matter.

The design pyramid: all six areas

Area	Easy meaning	Example: student portal
Interface	How users interact. Keep interaction consistent, guide tasks and provide clear navigation from different entry pages.	Clear forms, feedback and persistent navigation.
Aesthetic	Visual appeal suited to the target audience.	Readable fonts, spacing and colours.
Content	Structure content objects and their relationships; create text, graphics and media.	Organised notices, course details and results.
Navigation	How users move to content and features. Routes should match their needs.	Dashboard → Results → Semester.
Architecture	Overall organisation of content and functionality for presentation, navigation and interaction.	UI connected to services and data access.
Component	Detailed elements handling dynamic content, processing, data access and external integration.	A result-loading module or notification integration.

Three pairs students often confuse

• **Interface versus aesthetic:** easy interaction versus attractive appearance. A beautiful page can still be hard to use.

• **Content versus navigation:** what information is organised versus how users reach it.

• **Architecture versus component:** overall arrangement versus detailed working parts.

Two useful terms

Persistent navigation: important navigation remains available across relevant pages, even if users arrive through a direct link. **Dynamic content:** information that changes with current data or the user, such as personal marks.

Remember: WebApp design is not just decoration. Learn all six areas and one example of each.

12. Last-minute revision + practice

The whole unit in eight lines

• Design turns requirements into a blueprint; plan modules, calls, interfaces, algorithms and data.

• Goals: Firmness, Commodity, Delight. Diversify options, then converge on the best fit.

• Four models: Data/classes, Architecture, Interfaces, Components.

• FURPS: Functionality, Usability, Reliability, Performance, Supportability.

• Function-oriented = tasks. Object-oriented = objects with data + methods.

• **HIGH cohesion inside modules; LOW coupling between modules.**

• Three layers: Presentation → Business logic → Data access.

• UI: control, less memory, consistency. WebApp: interface, aesthetic, content, navigation, architecture, component.

Short-answer practice with answers

Question	Answer
What is software design?	Converting analysed requirements into a plan for construction.
What does CRC stand for?	Class–Responsibility–Collaborator.
What is stamp coupling in the source example?	Passing a whole User object when only email is needed.
Why is LoginViewModel cohesive?	Its methods all support login.
What is progressive disclosure?	Show high-level information first and reveal further detail when needed.
Reliability or performance: which includes throughput?	Performance. Reliability concerns dependable correct operation.

MCQs: try before reading the answer

1. Which combination is preferred?
A. High coupling/low cohesion • B. Low coupling/high cohesion
C. Low coupling/low cohesion • D. High coupling/high cohesion

2. Which layer applies application rules?
A. UI • B. Data access • C. Business logic • D. CSS

3. Ctrl+P best illustrates:
A. Stamp coupling • B. Intuitive shortcut • C. ORM • D. Inheritance

Answers: 1—B: focused modules with fewer dependencies. 2—C: business logic holds core rules. 3—B: P is a memorable association with Print.

Long-answer practice: use this structure

Compare cohesion and coupling: define both → compare inside/between → give preferred directions → explain LoginViewModel and Repository examples → give benefits.

Explain UI golden rules: name all three → explain their principles → add one example per rule. For WebApp design, name and explain all six areas.

Final check: Can you explain every bold term without looking? If not, return only to that topic—not the entire document.